



Mestrado em Eng. Informática

2025/26

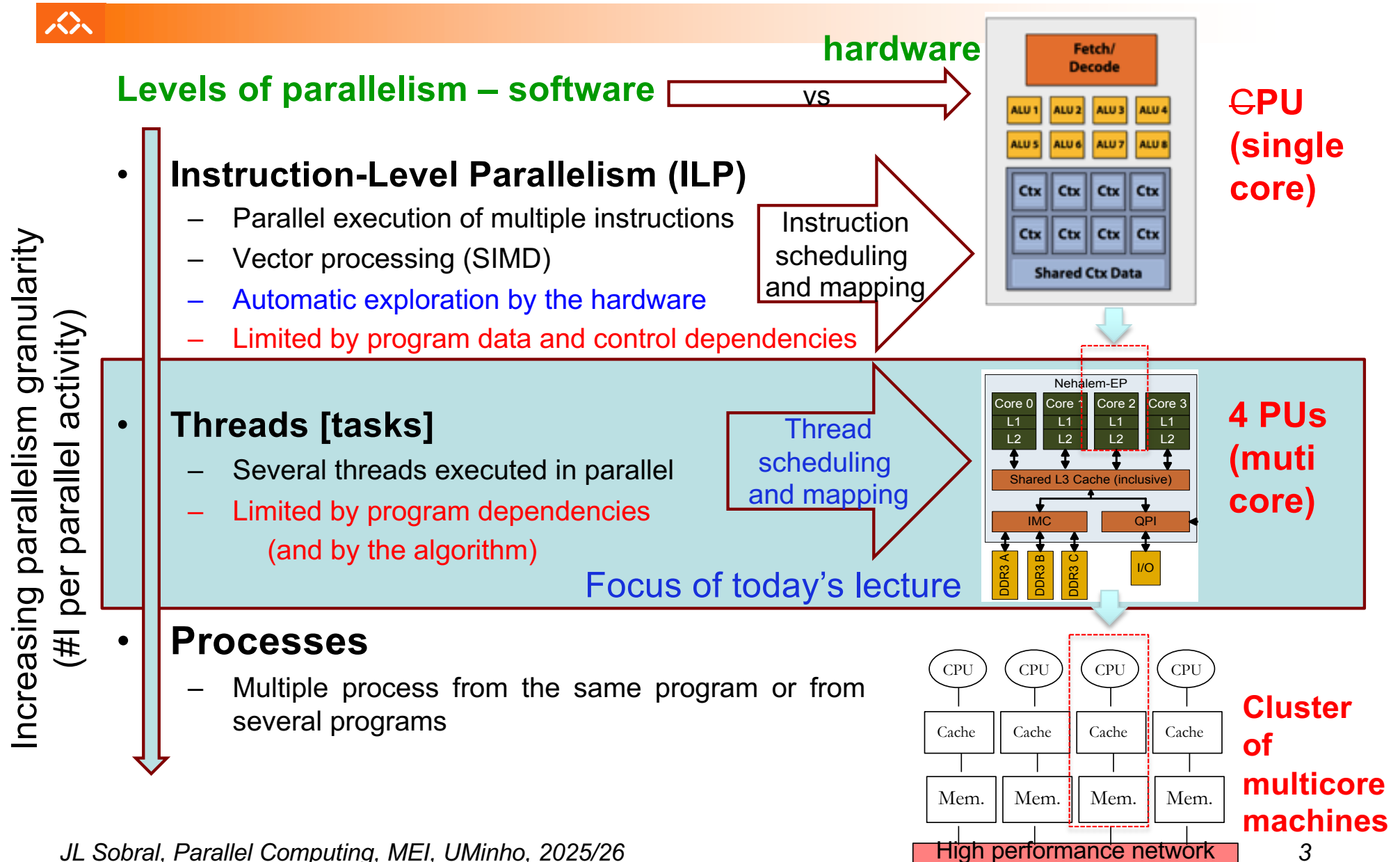
J.L. Sobral

Multithreading



- Task **granularity***: What is a "task"?
 - A single or multiple instructions (e.g., Instruction Level Parallelism)
 - One or several program statements
 - Call to functions of subroutines
 - A larger part of a program
 - A complete program (e.g., workflow)
-  **Increasing granularity**
(number of instructions,
#I, per parallel activity)
- *) A granule is a small particle of a substance, like a granule of sugar
- Strategies to explore parallelism
 - **Function parallelism** - runs different program functions in parallel
 - Speedup is limited by the number of "tasks" (i.e., functions)
 - **Data parallelism** – runs the same function across a set of data
 - **Better**: Parallelism scales as the data size increases

Thread-level parallelism



Thread scheduling

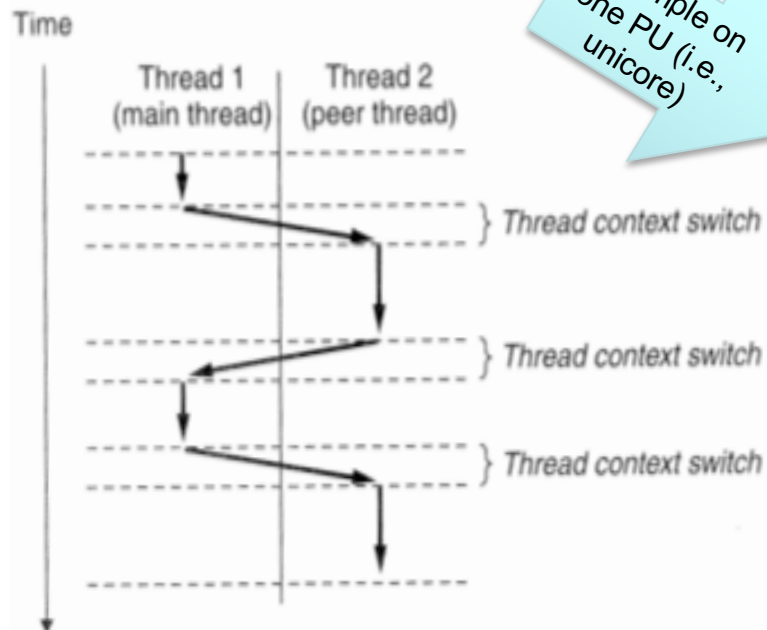
Logic vs physical parallelism



Thread Scheduling (on 1 PU)

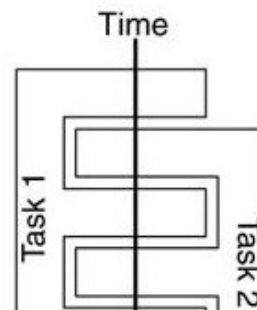
- The PU executes a set of threads, by scheduling blocks of instructions of each thread in time-slices (e.g. 50ms)
- It gives the illusion that threads are running in parallel

=> **Logic parallelism**

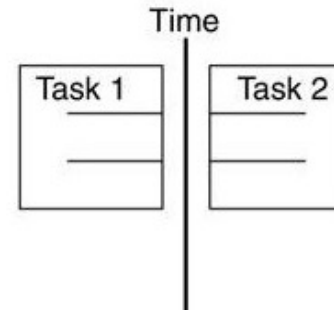


Example on one PU (i.e., uncore)

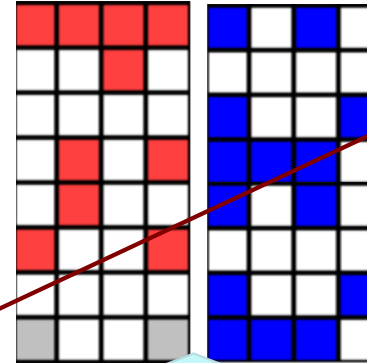
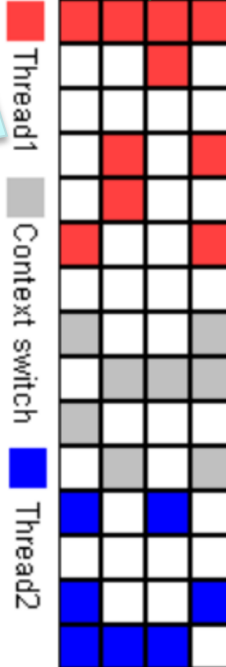
Concurrency



Parallelism



ILP => (4-way)



Example with 2 PUs

Physical parallelism

- At each clock cycle, instructions from multiple threads are running in parallel (on the same or in different PUs)

On current multicores, many threads are **scheduled on multiple PUs**

This scheduling and mapping has a **cost**:

- Thread creation
- Context switch
- Others (later...)

Scheduling result varies from execution to execution

Unicore Multithreading

(hardware-based thread scheduling)

Executing multiple threads on a single core (PU)

- Share all resources but replicate registers, PC/IP, etc.
- Switching between threads might require saving current thread context and load new thread context (IP, etc...)

■ There are several ways to support multithreading

1. Coarse-grain multithreading [includes OS controlled]

- Only switch on long stall (e.g., L2-cache miss) [or at the end of the time-slice]
- Simplifies hardware, but doesn't hide short stalls

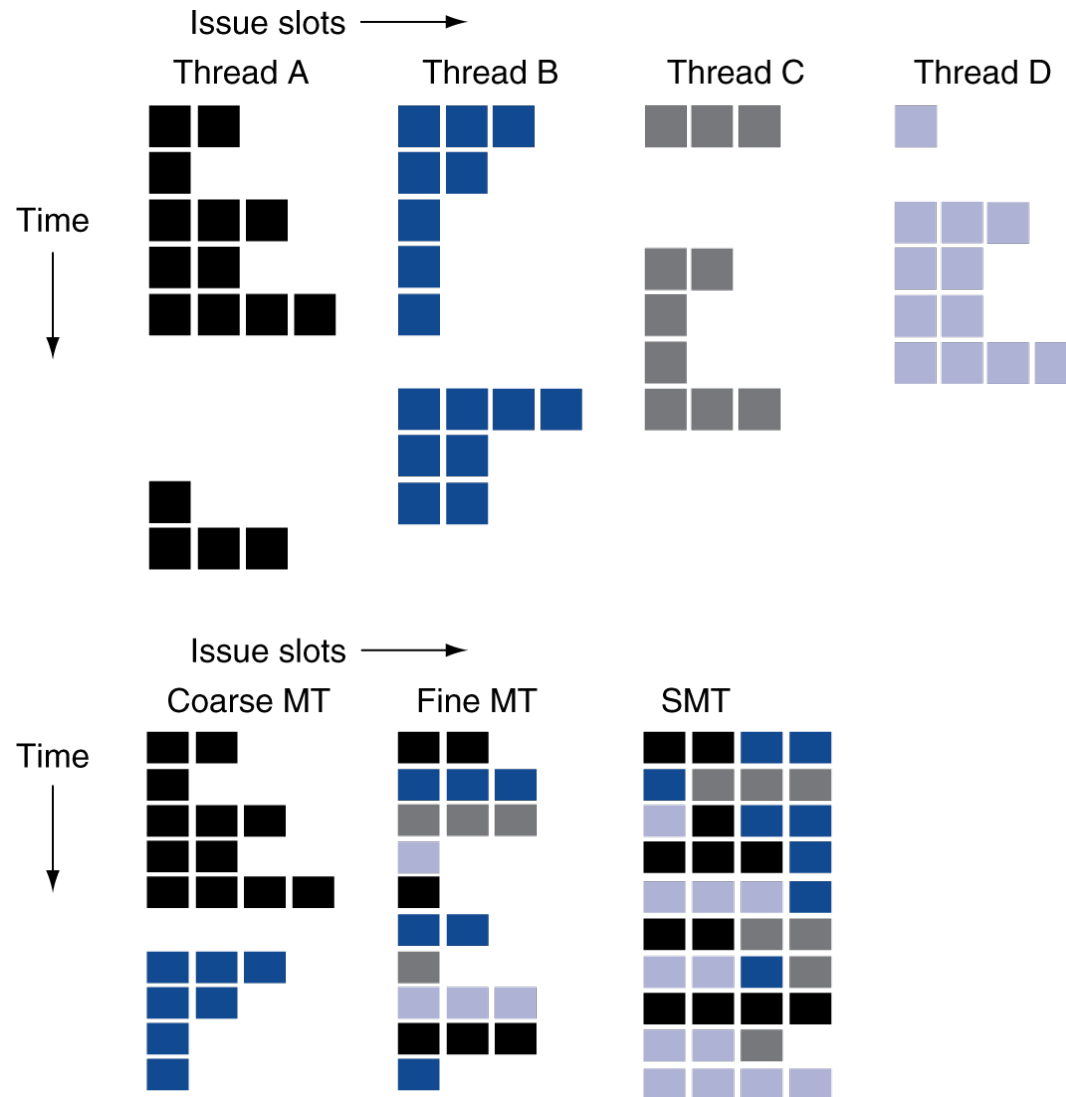
2. Fine-grain multithreading **=> on most GPU hardware**

- Switch threads after each clock cycle
- Interleave instruction execution
- If one thread stalls, others are executed

3. Simultaneous multithreading (*SMT - next slides...*)

Multithreading Example

Example with 4-way ILP
(support for execution of
4 instructions per clock
cycle)



Sharing superscalar resources

4-way superscalar
w/out MultiThreading

4-way superscalar & MultiThreading

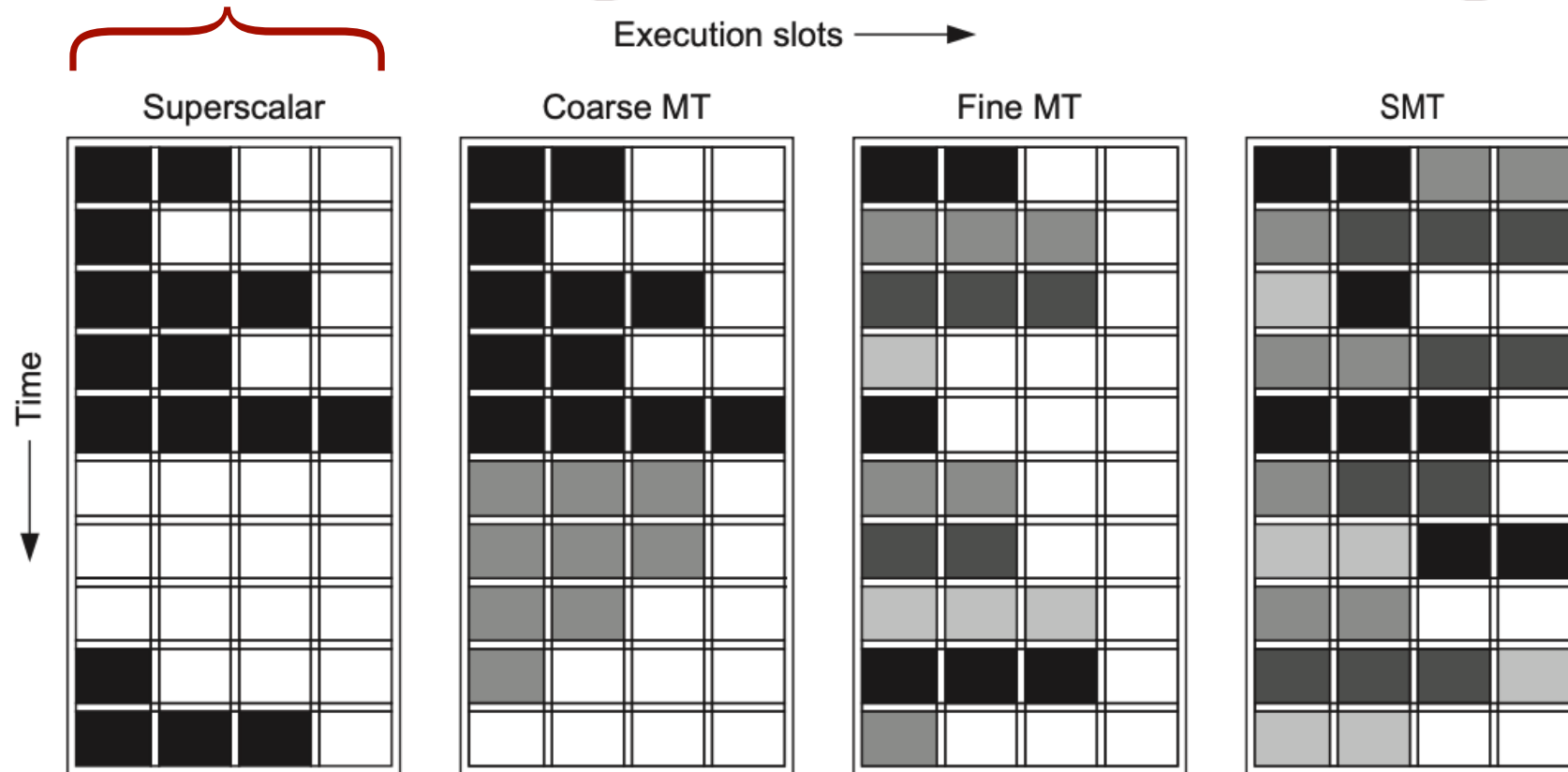
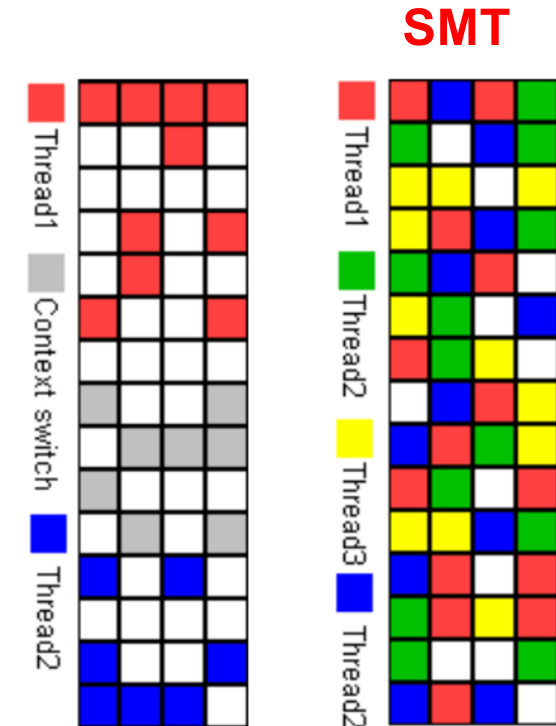


Figure 3.31 How four different approaches use the functional unit execution slots of a superscalar processor. The

3. Simultaneous Multithreading (SMT)

- In multiple-issue dynamically scheduled processor
 - Schedule instructions from multiple threads
 - Instructions from independent threads execute when function units are available
 - Within threads, dependencies handled by scheduling and register renaming
- Example: Intel from Pentium-4 HT
 - Two threads: duplicated registers, shared function units and caches



HT: Hyper-Threading, Intel trade mark for their SMT implementation

[NVIDIA] GPU Architecture

- Similarities to **vector machines**:

- Works well with data-level parallel problems
- Scatter-gather transfers
- Mask registers

- **Large register files**



Hardware-based thread scheduling

Registers are allocated to threads at creation

- fast context switch (& lower cost)

- constrains the maximum number of threads

This is fine-grain multithreading

- Differences:

- No scalar processor
- Uses multithreading to hide [memory] latency
- Has many functional units, as opposed to a few deeply pipelined units, behaving as a vector processor

GPU Organization (NVIDIA)

GPU Fine-grain multithreading
(1 clock cycle thread switch)

Warp scheduler

Hardware
thread
scheduler

A Multithreaded
SIMD Processor
(SM, Streaming
Multiprocessor)

← NVIDIA
terminology

Vector
processing

Large
register
file
(1Kx32)

